

AWS Concepts Explained

1. Virtual Machines (VMs) vs Servers

- Server: A physical machine that provides computing resources (CPU, RAM, storage, networking).
- VM (Virtual Machine): A software-based emulation of a computer system running on a physical server using a hypervisor (e.g., VMware ESXi, Hyper-V, KVM).
- Key point: A VM is not a server by itself, but behaves like one. A physical server can host multiple VMs, and each VM can act as a server if it runs server software.
- Analogy: A physical server is like an apartment building; each VM is like a flat inside it.

2. EC2 and ELB

- EC2 (Elastic Compute Cloud): A virtual server in AWS. You choose the OS, size, and storage. It's like renting a virtual machine from AWS.
- ELB (Elastic Load Balancer): Distributes incoming traffic across multiple EC2 instances, improving availability and scalability.
- Together: You host apps on EC2 and use ELB in front to distribute traffic.
- Analogy: EC2 = cooks, ELB = waiter distributing orders to the cooks.

3. Auto Scaling Group (ASG)

- ASG: Automatically manages EC2 instances.
- Scaling out: Adds instances when traffic is high.
- Scaling in: Removes instances when traffic is low.
- High availability: Replaces failed instances automatically.
- Example: ASG min=2, max=5. At peak, 5 instances run; after peak, back to 2.

4. Regions and Availability Zones (AZs)

- Region: Geographic area (e.g., Mumbai, Virginia). Isolated from other regions.
- AZ (Availability Zone): Separate data center(s) within a region with independent power, cooling, networking.
- Best practice: Deploy across multiple AZs for high availability.
- Analogy: Region = City, AZ = Different neighborhoods with power stations.

5. Edge Locations

- Edge Location: Smaller AWS site closer to users, used by CloudFront and Route 53.
- Purpose: Cache and deliver content with low latency.
- Analogy: Region = City, AZ = neighborhoods, Edge Location = Amazon delivery hubs.

6. Managed Services in AWS

- Definition: AWS services where AWS manages provisioning, patching, scaling, backups, high availability.
- Categories & Examples:
 - Compute: Lambda (serverless functions), Fargate (serverless containers), Elastic Beanstalk (application deployment)
 - Databases: RDS (relational DB), Aurora (high-performance relational DB), DynamoDB (NoSQL)
 - Storage & Data: S3 (object storage), EFS (elastic file system), Glacier (archival storage)
 - Networking & Delivery: CloudFront (CDN), Route 53 (DNS)

- Analytics & Big Data: Athena (query S3 with SQL), Redshift (data warehouse), EMR (Hadoop/Spark cluster)
- AI / ML: SageMaker (ML training & deployment), Rekognition (image/video analysis), Polly, Translate, Lex (speech, language, chatbots)
- Security & Monitoring: IAM (identity & access), GuardDuty (threat detection), CloudWatch (monitoring), AWS Shield / WAF (security)
- Analogy: EC2 = cook yourself, RDS = restaurant serves food.

7. Docker with AWS

- Docker: Packages apps into containers (self-contained units).
- AWS Integrations:
 - EC2: Install Docker, manage manually.
 - ECS: AWS's container orchestration.
 - EKS: Managed Kubernetes.
 - Fargate: Run containers without servers.
 - ECR: Docker image registry.
- Flow: Build → Push to ECR → Deploy via ECS/EKS/Fargate → ELB for traffic.
- Example: Node.js app → Docker → Push to ECR → Run on ECS Fargate.

8. Microservices Architecture in AWS

- Definition: Breaking a monolithic app into smaller, independent services.
- Compute: ECS/EKS/Fargate, Lambda.
- Networking: ELB, App Mesh, Cloud Map.
- API Management: API Gateway.
- Data: RDS, DynamoDB, S3.
- Communication: API Gateway (sync), SQS/SNS/EventBridge (async).
- Security: IAM, Cognito, Secrets Manager.
- Observability: CloudWatch, X-Ray.
- Example: React frontend (S3+CloudFront) → API Gateway → Auth (Lambda+DynamoDB), Orders (ECS+RDS), Payments (Lambda+SNS).

9. Serverless in AWS

- Definition: Servers exist, but AWS manages them. You focus only on code/config.
- Benefits: Auto scaling, no patching, pay per use, high availability.
- AWS Serverless Services:
 - Compute: Lambda, Fargate
 - API: API Gateway, EventBridge, SQS, SNS
 - Storage: S3, DynamoDB, Aurora Serverless, Athena
 - Orchestration: Step Functions, AppSync
 - Security: Cognito, Secrets Manager
 - Analytics: Kinesis, QuickSight
- Example: Serverless app → S3+CloudFront frontend, API Gateway, Lambda, DynamoDB, Cognito.
- Analogy: EC2 = manage servers, Serverless = AWS does everything, you only provide code.